

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-ab19af36c671af04d.jsonl`  
- SHA-256: `6d033ed74ef19be03a45ea2048912bbc866ce25222648240659198e0fbf04322`  
- Source modified: `2026-06-22T09:10:17+00:00`  
- Imported at: `2026-07-05T16:48:28+00:00`  
- project: `wf_d9866476-800`  
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:08:03.305Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

SHORT GOLDEN VIDEOS: read output/human_lovo_manifest.json and for each entry determine: cached trajectory CSV present? cached DB results present (output/*.db)? approx DURATION (infer from trajectory last-frame / fps, or note if unknown)? source video path + whether it's owned/local. ALSO check F:/ or G:/ referenced golden videos and the docs/GOLDEN_VIDEOS.md registry for owned, minors-free clips around 5-6 minutes. Rank the best 2-3 SHORT (<~6 min) owned candidates for a fast nightly full-pipeline run, noting for each whether GPU (WASB) is needed or cached results suffice.

2. assistant (2026-06-22T09:08:05.617Z)

I'll map this repo and identify golden video candidates for a nightly regression test. Let me start with a systematic exploration.

3. user (2026-06-22T09:08:06.591Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer  
On branch serving/exposure-safety-preflight  
Your branch is based on 'origin/serving/exposure-safety-preflight', but the upstream is gone.  
(use "git branch --unset-upstream" to fixup)
```

```
Untracked files:  
(use "git add <file>..." to include in what will be committed)  
  artifacts/  
  docs/KHELSUTRA_PRODUCT_OVERVIEW.md  
  docs/PACKAGING_PLAN.md
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

4. assistant (2026-06-22T09:08:08.789Z)

Now let me check the worktree and examine key files:

5. user (2026-06-22T09:08:09.512Z)

```
total 100  
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 .
```

```
drwxr-xr-x 1 avidu 197609 0 Jun 22 09:12 ..
drwxr-xr-x 1 avidu 197609 0 Jun 15 22:19 agent-a771ff1d85e18f2ed
drwxr-xr-x 1 avidu 197609 0 Jun 14 16:18 bhi-execpolicy
drwxr-xr-x 1 avidu 197609 0 Jun 17 16:57 boxhill17
drwxr-xr-x 1 avidu 197609 0 Jun 18 07:09 gap-doc
drwxr-xr-x 1 avidu 197609 0 Jun 16 11:31 gf-crossos
drwxr-xr-x 1 avidu 197609 0 Jun 16 10:34 golden-fixtures
drwxr-xr-x 1 avidu 197609 0 Jun 17 13:53 hardchop
drwxr-xr-x 1 avidu 197609 0 Jun 22 14:36 nightly-e2e
drwxr-xr-x 1 avidu 197609 0 Jun 17 20:01 pr-stitch
drwxr-xr-x 1 avidu 197609 0 Jun 18 12:36 recdoc
drwxr-xr-x 1 avidu 197609 0 Jun 18 10:41 resweep
drwxr-xr-x 1 avidu 197609 0 Jun 16 09:41 tier1-windowing
drwxr-xr-x 1 avidu 197609 0 Jun 16 11:24 w2-safeguard
```

6. user (2026-06-22T09:08:11.635Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\nightly-e2e'

7. assistant (2026-06-22T09:08:13.437Z)

Let me navigate to the right directory and check the key files:

8. user (2026-06-22T09:08:14.120Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e
total 207
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609   611 Jun 22 14:36 .coveragerc
-rw-r--r-- 1 avidu 197609    87 Jun 22 14:36 .git
-rw-r--r-- 1 avidu 197609   947 Jun 22 14:36 .gitattributes
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 .github
-rw-r--r-- 1 avidu 197609  1117 Jun 22 14:36 .gitignore
-rw-r--r-- 1 avidu 197609   525 Jun 22 14:36 CLAUDE.md
-rw-r--r-- 1 avidu 197609 35347 Jun 22 14:36 README.md
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 backend
-rw-r--r-- 1 avidu 197609  2735 Jun 22 14:36 config.json
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 deploy
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 docs
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 eval_baselines
-rw-r--r-- 1 avidu 197609  2344 Jun 22 14:36 mypy.ini
-rw-r--r-- 1 avidu 197609  4525 Jun 22 14:36 pyproject.toml
-rw-r--r-- 1 avidu 197609   245 Jun 22 14:36 pytest.ini
-rw-r--r-- 1 avidu 197609   642 Jun 22 14:36 requirements.txt
-rw-r--r-- 1 avidu 197609   884 Jun 22 14:36 ruff.toml
-rwxr-xr-x 1 avidu 197609  2055 Jun 22 14:36 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5620 Jun 22 14:36 run_eval.py
-rw-r--r-- 1 avidu 197609  4356 Jun 22 14:36 run_regression.py
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 scratch
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 scripts
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 tests
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 tools
drwxr-xr-x 1 avidu 197609      0 Jun 22 14:36 training
```

9. assistant (2026-06-22T09:08:16.175Z)

Now let me read the human_lovo_manifest and related files:

10. user (2026-06-22T09:08:17.864Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/boxhill17/docs/GOLDEN_VIDEOS.md
```

```
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/gap-  
doc/docs/GOLDEN_VIDEOS.md  
/c/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/nightly-e2e/docs/data_pipeline/GOLDEN_VIDEOS.md  
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/pr-  
stitch/docs/GOLDEN_VIDEOS.md  
/c/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/recdoc/docs/GOLDEN_VIDEOS.md  
/c/Users/avidu/Projects/badminton-highlight-  
indexer/.claude/worktrees/resweep/docs/GOLDEN_VIDEOS.md  
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/data_pipeline/GOLDEN_VIDEOS.md  
/c/Users/avidu/Projects/badminton-highlight-indexer/output/human_lovo_manifest.json
```

11. assistant (2026-06-22T09:08:19.671Z)

Perfect! Let me read these key files:

12. user (2026-06-22T09:08:20.069Z)

```
1      [  
2      {  
3          "name": "adarsh_avi_singles",  
4          "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",  
5          "fps": 59.94,  
6          "frame_width": 1920,  
7          "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh  
and Avi.rallies.csv"  
8      },  
9      {  
10         "name": "kushagra_singles",  
11         "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/kushagra_traj.csv",  
12         "fps": 59.94,  
13         "frame_width": 1920,  
14         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden  
Labelled/KushagraYashAviFirstVideo.rallies.csv"  
15     },  
16     {  
17         "name": "largetest_doubles",  
18         "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/largetestvideo_traj.csv",  
19         "fps": 29.97,  
20         "frame_width": 1920,  
21         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden  
Labelled/LargeTestVideo.rallies.csv"  
22     },  
23     {  
24         "name": "gbaaddy",  
25         "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/gbaaddy_traj.csv",  
26         "fps": 59.94,  
27         "frame_width": 1920,  
28         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden  
Labelled/gBaaddy20May2026.rallies.csv"  
29     },  
30     {  
31         "name": "testlarge_short",  
32         "trajectory": "C:/Users/avidu/Projects/Annotation  
Setup/Collect/Trajectories/testlarge_traj.csv",  
33         "fps": 29.97,  
34         "frame_width": 1920,  
35         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden  
Labelled/TestLargeVideo.rallies.csv"  
36     },  
37 ]
```

```

37     {
38         "name": "mahadevpura_singles",
39         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
40         "fps": 29.97,
41         "frame_width": 1920,
42         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv"
43     },
44     {
45         "name": "mahadevpura_2",
46         "fps": 59.94,
47         "frame_width": 1920.0,
48         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_2.rallies.csv",
49         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura2_june12_traj.csv"
50     },
51     {
52         "name": "GX010128",
53         "fps": 59.94,
54         "frame_width": 1920.0,
55         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010128.rallies.csv",
56         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX010128_june12_traj.csv"
57     },
58     {
59         "name": "mahadevpura_1",
60         "fps": 59.94,
61         "frame_width": 1920.0,
62         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_1.rallies.csv",
63         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura1_june12_traj.csv"
64     },
65     {
66         "name": "GX030094",
67         "fps": 29.97,
68         "frame_width": 1920.0,
69         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX030094.rallies.csv",
70         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX030094_june15_traj.csv"
71     },
72     {
73         "name": "Badminton_BXH_2",
74         "fps": 59.94,
75         "frame_width": 1920.0,
76         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/Badminton_BXH_2.rallies.csv",
77         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Badminton_BXH_2_june15_traj.csv"
78     },
79     {
80         "name": "GX010141",
81         "fps": 59.94,
82         "frame_width": 1920.0,
83         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010141.rallies.csv",
84         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX010141_june17_traj.csv"
85     },
86     {
87         "name": "GX010137",

```

```

88         "fps": 59.94,
89         "frame_width": 1920.0,
90         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010137.rallies.csv",
91         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX010137_june17_traj.csv"
92     },
93     {
94         "name": "Boxhill_Doubles",
95         "fps": 29.97,
96         "frame_width": 1920.0,
97         "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/Boxhill_Doubles.rallies.csv",
98         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/Boxhill_Doubles_june18_traj.csv"
99     },
100    {
101        "name": "GX020094",
102        "fps": 29.97,
103        "frame_width": 1920.0,
104        "labels": "C:/Users/avidu/Projects/badminton-highlight-
indexer/output/GX020094.rallies.csv",
105        "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/GX020094_june18_traj.csv"
106    }
107 ]

```

13. assistant (2026-06-22T09:08:22.149Z)

Now let me check trajectory files and the GOLDEN_VIDEOS registry:

14. user (2026-06-22T09:08:22.970Z)

```

1      # Golden-labeled video registry
2
3      **Purpose.** The single source of truth for every **golden** (human rally-boundary-
labeled) video ? what
4      it is, where the source proxy lives, how many rallies, and whether it's ingested into
the eval corpus.
5      Use this to **isolate the golden videos into a dedicated location** and to keep the
growing golden set
6      auditable. Updated whenever a new video is labeled + ingested.
7
8      > **This is the GROWING GOLDEN-finalized set** in the dual-eval-set discipline (see
9      > [QUALITY_ITERATIONS.md](../QUALITY_ITERATIONS.md), the 2026-06-16 capstone + `eval-
dual-set-regression`):
10     > a **RAW eval set** (currently-bad videos) + this **golden-finalized eval set**
(promoted when at-par) +
11     > a separate **golden TRAINING set**. Every quality launch must not regress on either
eval set.
12     > Privacy-safe, video-free *shipping* of these =
[GOLDEN_REGRESSION_FIXTURES.md](../GOLDEN_REGRESSION_FIXTURES.md).
13
14     ## Contracts / conventions
15     - **Labels:** rally-annotator CSV
(`rally_number,start_time,end_time,ending_reason,sport,shots_count`,
16     decimal seconds) ? ingested by `backend/eval/gt_loader.py` (reads
`start_time`/`end_time`).
17     - **Alignment is mandatory before trusting any score.** The label timebase MUST match
the windowed proxy
18     (same fps + frame count). Verify with `ffprobe` (the `largetest_doubles` near-miss is
why) ? ideally
19     annotate the *exact* proxy the detector ran on (then `video_id` = MD5 matches and
alignment is free).
20     - **`video_id`** = MD5 of the proxy (`backend/utils/hashing.py::compute_video_id`); it

```

```

keys the WASB
21     trajectory cache, the Gemini reference rows, and the golden features.
22     - **Ingestion** = copy the trajectory + labels to durable paths, write
`output/<name>_golden_features.json`
23     (`gts` + candidates), append a row to `output/human_lovo_manifest.json`.
(`scratch/at_par.py` scores a
24     single video vs Gemini + golden; `backend/eval/rally_seg_eval.py` runs the whole
corpus.)
25     - **Guardrails:** owned footage only; **minors excluded**; labels never derived from a
paid LLM (Gemini is
26     inference/reference only). Commercial-clean.
27
28     ## Ingested golden corpus ? `output/human_lovo_manifest.json` (n = 15)
29
30     | # | name | sport | rallies | fps | source proxy | video_id | labeled |
31     |---|---|---|---|---|---|---|---|
32     | 1 | adarsh_avi_singles | badminton (S) | 96 | 59.94 | (original 6) | ? | pre-06-16 |
33     | 2 | kushagra_singles | badminton (S) | 32 | 59.94 | (original 6) | ? | pre-06-16 |
34     | 3 | largetest_doubles | badminton (D) | 49 | 29.97 | (original 6) | ? | pre-06-16 |
35     | 4 | gbaaddy | badminton | 48 | 59.94 | (original 6) | ? | pre-06-16 |
36     | 5 | testlarge_short | badminton | 6 | 29.97 | (original 6) | ? | pre-06-16 |
37     | 6 | mahadevpura_singles | badminton (S) | 31 | 29.97 | (original 6) | ? | pre-06-16 |
38     | 7 | mahadevpura_2 | badminton (multicourt) | 40 | 59.94 | `?/Roora Request - June
15/[Done] Video 3 ?/mahadevpura-2_proxy.mp4` | `038e0e0915a2` | 2026-06-16 |
39     | 8 | GX010128 | badminton (multicourt) | 52 | 59.94 | `?/Golden Label Videos Request -
June 15/[Done] Video 13 ?/GX010128.MP4` | `db84f3e65318` | 2026-06-16 |
40     | 9 | mahadevpura_1 | badminton (S, continuous) | 10 | 59.94 | `?/[Done] Video 14 -
Badminton/mahadevpura-1.MP4` | `38bef75d78cb` | 2026-06-17 |
41
42     ## Labeled, ingestion in progress (WASB trajectory extracting)
43
44     | name | sport | rallies | fps | source proxy | status |
45     |---|---|---|---|---|---|
46     | GX030094 | badminton | 41 | **29.97** | `?/Golden Label ?/[Done] Video 6 -
Badminton/GX030094_proxy.mp4` | **ingested** (in `human_lovo_manifest.json`) |
47     | Badminton BXH 2 | badminton (multicourt) | 44 | 59.94 | `?/[Done] Video 10 - Badminton
Multicourt/Badminton BXH 2_proxy.mp4` | **ingested** (in `human_lovo_manifest.json`) |
48
49     ## Source folder & not-yet-labeled
50     - **Annotation root:** `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos
Request - June 15\`
51     (+ the earlier `Roora Request - June 15\`). `[Done]` prefix in a dir name = golden-
labeled.
52     - **Labeled badminton dirs:** Video 3 (=mahadevpura-2), 6 (=GX030094), 10 (=BXH 2), 13
(=GX010128, a
53     re-label of #8), 14 (=mahadevpura-1). **Video 13 is a duplicate of #8** (already
ingested).
54     - **Not yet labeled / other sports:** Video 1, 4, 7, 9, 12 (badminton, pending); Video
2, 8 (table tennis);
55     Video 5, 11 (pickleball). Badminton is the current focus (alpha customers);
TT/pickleball labeled later.
56
57     ## How to isolate the golden videos
58     Each row's **source proxy** column is the file to copy into a dedicated golden-videos
location. The labels
59     live beside the proxy (`<name>.rallies.csv`) and a durable copy is in
`output/<name>.rallies.csv`; the
60     trajectory is in `Collect/Trajectories/` (path in the manifest). Keep this table in sync
on every new ingest.
61

```

15. assistant (2026-06-22T09:08:25.201Z)

Now let me check the trajectory files to determine their sizes/durations:

16. user (2026-06-22T09:08:28.433Z)

```
=== C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/adarsh_avi_full_traj.csv ===  
84480,0,-inf,-inf
```

```
=== C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/kushagra_traj.csv ===  
32232,0,-inf,-inf
```

```
=== C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/testlarge_traj.csv ===  
3019,1,1705.3294372558594,397.50125885009766
```

```
=== C:/Users/avidu/Projects/Annotation Setup/Collect/Trajectories/mahadevpura1_june12_traj.csv  
===  
10447,1,249.77148056030273,610.7673454284668
```

17. assistant (2026-06-22T09:08:31.380Z)

Let me check for database results and verify cached trajectory and DB files:

18. user (2026-06-22T09:08:32.405Z)

```
-rw-r--r-- 1 avidu 197609 236K Jun 19 17:27 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/19june.db  
-rw-r--r-- 1 avidu 197609 36K May 19 23:07 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/badminton_indexer.db  
-rw-r--r-- 1 avidu 197609 88K Jun 19 18:36 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/gemini_19june.db  
-rw-r--r-- 1 avidu 197609 76K Jun 18 13:47 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/gemini_long.db  
-rw-r--r-- 1 avidu 197609 84K Jun 16 02:27 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/local_compare.db  
-rw-r--r-- 1 avidu 197609 112K May 26 03:53 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/phase3_yolo.db  
-rw-r--r-- 1 avidu 197609 252K Jun 17 14:41 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/sports_indexer.db  
-rw-r--r-- 1 avidu 197609 24K May 20 12:04 /c/Users/avidu/Projects/badminton-highlight-  
indexer/output/test_indexer.db
```

19. assistant (2026-06-22T09:08:34.789Z)

Now let me check if remote golden videos exist on F: or G::

20. user (2026-06-22T09:08:35.806Z)

(Bash completed with no output)

21. assistant (2026-06-22T09:08:40.859Z)

Let me check if the golden videos folder exists in a different location:

22. user (2026-06-22T09:08:41.542Z)

(Bash completed with no output)

23. assistant (2026-06-22T09:08:43.689Z)

Let me check for cached trajectory CSVs in the output directory and verify file presence:

24. user (2026-06-22T09:08:45.337Z)

22

```
-rw-r--r-- 1 avidu 197609 819K Jun 18 11:28 Boxhill_Doubles_june18_traj.csv  
-rw-r--r-- 1 avidu 197609 1.5M Jun 16 19:04 GX010128_june12_traj.csv
```

```

-rw-r--r-- 1 avidu 197609 779K Jun 17 18:13 GX010137_june17_traj.csv
-rw-r--r-- 1 avidu 197609 758K Jun 17 17:23 GX010141_june17_traj.csv
-rw-r--r-- 1 avidu 197609 594K Jun 18 11:53 GX020094_june18_traj.csv
-rw-r--r-- 1 avidu 197609 686K Jun 17 14:15 GX030094_june15_traj.csv
-rw-r--r-- 1 avidu 197609 2.4M Jun 8 18:24 adarsh_avi_full_traj.csv
-rw-r--r-- 1 avidu 197609 1.1M Jun 8 14:24 adarsh_avi_traj.csv
-rw-r--r-- 1 avidu 197609 1.1M Jun 6 20:53 adarsh_traj.csv
-rw-r--r-- 1 avidu 197609 1.7M Jun 8 19:11 gbaaddy_traj.csv
-rw-r--r-- 1 avidu 197609 1.2M Jun 8 18:42 kushagra_traj.csv
-rw-r--r-- 1 avidu 197609 678K Jun 8 21:02 largetestvideo_traj.csv
-rw-r--r-- 1 avidu 197609 449K Jun 16 00:58 mahadevpura1_june12_traj.csv
-rw-r--r-- 1 avidu 197609 1.4M Jun 16 02:26 mahadevpura2_june12_traj.csv
-rw-r--r-- 1 avidu 197609 409K Jun 9 10:40 mahadevpura_traj.csv
-rw-r--r-- 1 avidu 197609 142K Jun 9 18:10 pickleball_traj.csv
drwxr-xr-x 1 avidu 197609 0 Jun 10 12:56 roora
-rw-r--r-- 1 avidu 197609 587K Jun 6 18:01 sample_long_traj.csv
-rw-r--r-- 1 avidu 197609 95K Jun 9 18:13 tabletennis_traj.csv
-rw-r--r-- 1 avidu 197609 95K Jun 6 22:20 testlarge_traj.csv

```

25. assistant (2026-06-22T09:08:49.146Z)

Now let me compute the duration for each video based on last frame and fps:

26. user (2026-06-22T09:08:56.137Z)

adarsh_avi_singles	59.94 fps 1409.41 sec (23.49 min)
kushagra_singles	59.94 fps 537.74 sec (8.96 min)
largetest_doubles	29.97 fps 768.77 sec (12.81 min)
gbaaddy	59.94 fps 871.40 sec (14.52 min)
testlarge_short	29.97 fps 100.73 sec (1.68 min)
mahadevpura_singles	29.97 fps 447.71 sec (7.46 min)
mahadevpura_2	59.94 fps 615.62 sec (10.26 min)
GX010128	59.94 fps 755.71 sec (12.60 min)
mahadevpura_1	59.94 fps 174.29 sec (2.90 min)
GX030094	29.97 fps 707.24 sec (11.79 min)
Badminton_BXH_2	59.94 fps 728.96 sec (12.15 min)
GX010141	59.94 fps 385.05 sec (6.42 min)
GX010137	59.94 fps 451.23 sec (7.52 min)
Boxhill_Doubles	29.97 fps 768.74 sec (12.81 min)
GX020094	29.97 fps 768.74 sec (12.81 min)

27. assistant (2026-06-22T09:08:58.052Z)

Excellent! Now let me check the nightly regression script and configuration:

28. user (2026-06-22T09:08:59.497Z)

```

1      """Nightly rally-quality regression tripwire.
2
3      Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5      the headline metrics against a FROZEN baseline
(``eval_baselines/nightly_baseline.json``).
6      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10     + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it

```

```

12     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22     the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24     permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     -----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38         python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42         python scripts/nightly_regression.py
43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48     over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52     but no regression was found on the comparable intersection.
53     """
54     from __future__ import annotations
55
56     import argparse
57     import datetime as _dt
58     import json
59     import os

```

```

60     import sys
61     from dataclasses import dataclass, field, replace
62     from typing import Any, Dict, List, Optional, Tuple
63
64     # ----- #
65     # Paths (resolved relative to the repo root, so the script is cwd-robust).
66     # ----- #
67     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68     if REPO_ROOT not in sys.path:
69         sys.path.insert(0, REPO_ROOT) # importable as `python
scripts/nightly_regression.py` from anywhere
70     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74     DEFAULT_IOU = 0.5
75
76     # ----- #
77     # What we gate on (a tripwire ? focused, not every metric).
78     # * HIGHER-better aggregates regress when they DROP beyond `metric_tol`.
79     # * merge_rate (lower-better) regresses when it RISES beyond `rate_tol`.
80     # * per-video split_count / merge_count must NOT increase beyond `over_seg_tol`
81     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83     # ----- #
84     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89     AGG_SNAPSHOT: Tuple[str, ...] = (
90         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
"dend_abs_median",
92     )
93     #: Per-video metrics snapshotted into the baseline.
94     PV_SNAPSHOT: Tuple[str, ...] = (
95         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
"num_pred", "num_gt",
96     )
97
98     DEFAULT_METRIC_TOL = 0.005 # F1-like drop that counts as a regression (absorbs float
jitter)
99     DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression
100     DEFAULT_OVER_SEG_TOL = 0 # per-video split/merge increase allowed (strict)
101
102
103     # ----- #
104     # The two tracked configs.
105     # ----- #
106     @dataclass(frozen=True)
107     class NightlyConfig:
108         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110         name: str
111         preset: Any # backend.eval.windowing.WindowingPreset
112         min_crossings: int = 0
113         note: str = ""
114
115
116     def tracked_configs() -> List[NightlyConfig]:
117         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).

```

```

118
119     All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120     min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122     * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123     point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124     DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125     knob.
126     * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127     the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128     what we expect?" stays answerable even if a future change moves DEFAULT.
129     * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130     the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
131     dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132     launch-time value). It is the reference DEFAULT used to be before the flip.""
133     from backend.eval.windowing import SERVED
134
135     default = NightlyConfig(
136         name="default", preset=SERVED, min_crossings=0,
137         note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138     )
139     milestone = NightlyConfig(
140         name="milestone",
141         preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142             inactivity_end=1.5, inactivity_rally_thresh=0.20,
143             inactivity_min_density=0.30),
144         min_crossings=0,
145         note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
146     )
147     baseline_off = NightlyConfig(
148         name="baseline_off",
149         preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
hard_cap=False,
150             inactivity_end=0.0, inactivity_rally_thresh=0.08,
inactivity_min_density=0.34,
151             blob_fallback=False, blob_factor=4.0),
152         min_crossings=0,
153         note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
154     )
155     return [default, milestone, baseline_off]
156
157
158     # ----- #
159     # Tolerances bundle.
160     # ----- #
161     @dataclass(frozen=True)
162     class Tolerances:
163         metric_tol: float = DEFAULT_METRIC_TOL
164         rate_tol: float = DEFAULT_RATE_TOL
165         over_seg_tol: int = DEFAULT_OVER_SEG_TOL
166
167     def to_dict(self) -> Dict[str, Any]:
168         return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,

```

```

169         "over_seg_tol": self.over_seg_tol}
170
171
172     # ----- #
173     # Pure summary + comparison core (no I/O ? unit-testable).
174     # ----- #
175     def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
176         """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
177
178         Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
179         which is seeded but irrelevant to the gate) and per-video RAW metric values."""
180         agg = eval_result.get("aggregate", {}) or {}
181         aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
182         per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
183         return {
184             "n": int(agg.get("n", len(eval_result.get("rows", [])))),
185             "preset": eval_result.get("preset"),
186             "min_crossings": eval_result.get("min_crossings", 0),
187             "iou": eval_result.get("iou", DEFAULT_IOU),
188             "aggregate": aggregate,
189             "per_video": per_video,
190         }
191
192
193     @dataclass
194     class Finding:
195         """One comparison outcome line (regression / improvement / stale note)."""
196
197         config: str
198         scope: str          # "aggregate" | "per_video" | "config"
199         metric: str
200         kind: str           # "regression" | "improvement" | "stale"
201         baseline: Optional[float] = None
202         current: Optional[float] = None
203         video: Optional[str] = None
204
205         @property
206         def delta(self) -> Optional[float]:
207             if self.baseline is None or self.current is None:
208                 return None
209             return self.current - self.baseline
210
211         def message(self) -> str:
212             loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
213             if self.kind == "stale":
214                 return f"[STALE] {self.config}: {self.metric}"
215             d = self.delta
216             ds = f"{d:+.4f}" if d is not None else "?"
217             b = "?" if self.baseline is None else f"{self.baseline:.4f}"
218             c = "?" if self.current is None else f"{self.current:.4f}"
219             tag = "REGRESSION" if self.kind == "regression" else "improvement"
220             return f"[{tag}] {loc}: {b} ? {c} ({ds})"
221
222
223     @dataclass
224     class ConfigComparison:
225         config: str
226         status: str = "ok"          # "ok" | "regression" | "stale"
227         findings: List[Finding] = field(default_factory=list)
228         added_videos: List[str] = field(default_factory=list)
229         removed_videos: List[str] = field(default_factory=list)

```

```

230         aggregate_gated: bool = True # False when preset/corpus changed ? aggregate not
comparable
231
232     @property
233     def regressions(self) -> List[Finding]:
234         return [f for f in self.findings if f.kind == "regression"]
235
236     @property
237     def improvements(self) -> List[Finding]:
238         return [f for f in self.findings if f.kind == "improvement"]
239
240
241     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242         """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
243         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
244             return base_preset == cur_preset
245         keys = set(base_preset) | set(cur_preset)
246         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
247
248
249     def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                       tols: Tolerances) -> ConfigComparison:
251         """Compare one config's current summary to its baseline summary.
252
253         * If the preset definition or the net-crossings gate changed, the whole config is
STALE
254         (numbers aren't comparable) ? flag it, skip gating.
255         * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
256         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
257         INTERSECTION (a code regression on a shared video must still trip the wire).
258         """
259         cmp = ConfigComparison(config=name)
260
261         # 1) Config-definition drift ? stale, not gateable.
262         if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
263             or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
264             cmp.status = "stale"
265             cmp.aggregate_gated = False
266             cmp.findings.append(Finding(config=name, scope="config", metric="preset",
kind="stale"))
267         return cmp
268
269         base_pv = base_sum.get("per_video", {})
270         cur_pv = cur_sum.get("per_video", {})
271         base_names, cur_names = set(base_pv), set(cur_pv)
272         cmp.added_videos = sorted(cur_names - base_names)
273         cmp.removed_videos = sorted(base_names - cur_names)
274         corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
275
276         # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
277         if corpus_changed:
278             cmp.aggregate_gated = False
279             cmp.status = "stale"
280             cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
281         else:
282             base_agg = base_sum.get("aggregate", {})
283             cur_agg = cur_sum.get("aggregate", {})
284             for m in GATED_HIGHER:
285                 b, c = base_agg.get(m), cur_agg.get(m)
286                 if b is None or c is None:

```

```

287         continue
288     if c < b - tols.metric_tol:
289         cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
290     elif c > b + tols.metric_tol:
291         cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
292     for m in GATED_LOWER:
293         b, c = base_agg.get(m), cur_agg.get(m)
294         if b is None or c is None:
295             continue
296         if c > b + tols.rate_tol:
297             cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
298         elif c < b - tols.rate_tol:
299             cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
300
301     # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
302     regressions).
303     for vid in sorted(base_names & cur_names):
304         for m in PER_VIDEO_GUARD:
305             b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
306             if b is None or c is None:
307                 continue
308             if c > b + tols.over_seg_tol:
309                 cmp.findings.append(Finding(name, "per_video", m, "regression",
310 float(b), float(c), video=vid))
311
312     if cmp.regressions:
313         cmp.status = "regression"
314     elif cmp.status != "stale":
315         cmp.status = "ok"
316     return cmp
317
318 @dataclass
319 class NightlyResult:
320     comparisons: List[ConfigComparison] = field(default_factory=list)
321     skipped: Dict[str, List[str]] = field(default_factory=dict) # config -> manifest
322     videos not scored
323
324     @property
325     def has_regression(self) -> bool:
326         return any(c.status == "regression" for c in self.comparisons)
327
328     @property
329     def has_stale(self) -> bool:
330         return any(c.status == "stale" for c in self.comparisons)
331
332     def overall_status(self) -> str:
333         if self.has_regression:
334             return "REGRESSION"
335         if self.has_stale:
336             return "STALE"
337         return "PASS"
338
339     def exit_code(self) -> int:
340         if self.has_regression:
341             return 1
342         if self.has_stale:
343             return 4
344         return 0
345
346     def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
347                     tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
348         """Compare every tracked config's current summary to the baseline summary."""

```

```

348     res = NightlyResult(skipped=skipped or {})
349     base_cfgs = baseline.get("configs", {})
350     cur_cfgs = current.get("configs", {})
351     for name in cur_cfgs:
352         if name not in base_cfgs:
353             cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
354             cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
355             res.comparisons.append(cmp)
356             continue
357             res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
358     return res
359
360
361     # ----- #
362     # Report formatting (pure).
363     # ----- #
364     def _fmt(v: Optional[float]) -> str:
365         return "?" if v is None else f"{v:.4f}"
366
367
368     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
369                      result: NightlyResult, tols: Tolerances, date: str) -> str:
370         L: List[str] = []
371         status = result.overall_status()
372         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
373         L.append(f"# Nightly rally-quality regression ? {date}")
374         L.append("")
375         L.append(f"***Status: {badge}**   exit code {result.exit_code()}")
376         L.append("")
377         L.append(f"- HEAD: `{current.get('git_commit', '?')}`   baseline: "
378                 f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')}")
379         L.append(f"- Corpus manifest: `{current.get('manifest', '?')}`   .
IoU={current.get('iou', DEFAULT_IOU)}")
380         L.append(f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
381                 f"per-video over-seg ±{tols.over_seg_tol}")
382         L.append("")
383
384         # Up-front regression / stale summary.
385         regs = [f for c in result.comparisons for f in c.regressions]
386         if regs:
387             L.append("## ? Regressions")
388             L.append("")
389             for f in regs:
390                 L.append(f"- {f.message()}")
391             L.append("")
392         if result.has_stale:
393             L.append("## ? Stale / not comparable")
394             L.append("")
395             for c in result.comparisons:
396                 if c.status == "stale":
397                     if c.added_videos:
398                         L.append(f"- `{c.config}`: corpus added {c.added_videos}")
399                     if c.removed_videos:
400                         L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
401                     if not c.added_videos and not c.removed_videos:
402                         L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
403             L.append(f"- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
404             L.append("")
405

```

```

406         # Skipped (no silent caps).
407         skipped_any = {k: v for k, v in result.skipped.items() if v}
408         if skipped_any:
409             L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
410             L.append("")
411             for cfg, vids in skipped_any.items():
412                 L.append(f"- {cfg}: {vids}")
413             L.append("")
414
415         # Per-config detail.
416         cur_cfgs = current.get("configs", {})
417         base_cfgs = baseline.get("configs", {})
418         for cmp in result.comparisons:
419             cur = cur_cfgs.get(cmp.config, {})
420             base = base_cfgs.get(cmp.config, {})
421             L.append(f"## Config {cmp.config} ? {cur.get('note', '')}")
422             L.append("")
423             L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})_")
424             L.append("")
425             cagg = cur.get("aggregate", {})
426             bagg = base.get("aggregate", {})
427             L.append("| metric | baseline | current | ? | gated |")
428             L.append("|---|---|---|---|---|")
429             for m in AGG_SNAPSHOT:
430                 b, c = bagg.get(m), cagg.get(m)
431                 d = (c - b) if (b is not None and c is not None) else None
432                 gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
433                 L.append(f"| {m} | {fmt(b)} | {fmt(c)} | {fmt(d)} | {gated} |")
434             L.append("")
435             # Per-video over-seg guard table.
436             cpv = cur.get("per_video", {})
437             bpv = base.get("per_video", {})
438             names = sorted(set(cpv) | set(bpv))
439             if names:
440                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
441                 L.append("|---|---|---|---|")
442                 for v in names:
443                     cb, cc = bpv.get(v, {}), cpv.get(v, {})
444                     tf = f"{fmt(cb.get('tol_f1'))}{fmt(cc.get('tol_f1'))}"
445                     sp = f"{cb.get('split_count', '?')}{cc.get('split_count', '?')}"
446                     mg = f"{cb.get('merge_count', '?')}{cc.get('merge_count', '?')}"
447                     flag = "?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
448                     L.append(f"| {v} {flag} | {tf} | {sp} | {mg} |")
449                 L.append("")
450                 imps = cmp.improvements
451                 if imps:
452                     L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
453                     for f in imps:
454                         L.append(f"- {f.message()}")
455                     L.append("")
456
457             L.append("----")
458             L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change ")
459             "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
460             return "\n".join(L)
461
462
463         # -----

```

```

464 # I/O shell.
465 # ----- #
466 def _git_commit() -> str:
467     import subprocess
468     try:
469         out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
470                             capture_output=True, text=True, timeout=10)
471         return out.stdout.strip() or "?"
472     except Exception:
473         return "?"
474
475
476 def _display_path(path: str) -> str:
477     """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
478     in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
479     Metadata only ? the comparison never reads it."""
480     norm = path.replace("\\", "/")
481     parent = os.path.basename(os.path.dirname(norm))
482     if parent == "output":
483         return "output/" + os.path.basename(norm)
484     try:
485         rel = os.path.relpath(path, REPO_ROOT)
486         if not rel.startswith(".."):
487             return rel.replace("\\", "/")
488     except ValueError:
489         pass
490     return norm
491
492
493 def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
494               iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
495     """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).
496
497     ``snapshot`` is the serialisable structure stored as / compared to the baseline.
498     ``skipped``
499     lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
500     surfaced in
501     the report so the corpus can't silently shrink."""
502     from backend.eval import rally_seg_eval
503
504     golden = rally_seg_eval.load_golden(manifest, features_dir)
505     manifest_names = [v["name"] for v in golden]
506     if not golden:
507         raise RuntimeError(f"no golden videos loaded from {manifest}")
508
509     snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
"configs": {}}
510     skipped: Dict[str, List[str]] = {}
511     scored_any = False
512     for cfg in configs:
513         result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
min_crossings=cfg.min_crossings)
514         summary = summarize_run(result)
515         summary["note"] = cfg.note
516         snapshot["configs"][cfg.name] = summary
517         scored = set(summary["per_video"])
518         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
519         scored_any = scored_any or bool(scored)
520     if not scored_any:
521         raise RuntimeError(
f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "

```

```

521         f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"")
522     return snapshot, skipped
523
524
525     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
526         if not os.path.exists(path):
527             return None
528         with open(path, encoding="utf-8") as fh:
529             return json.load(fh)
530
531
532     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
533         out = dict(snapshot)
534         out["generated_at"] = _dt.date.today().isoformat()
535         out["git_commit"] = _git_commit()
536         out["tolerances"] = tols.to_dict()
537         out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
538             "`python scripts/nightly_regression.py --update-baseline` after an
intended "
539             "improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed).")
540         os.makedirs(os.path.dirname(path), exist_ok=True)
541         tmp = path + ".tmp"
542         with open(tmp, "w", encoding="utf-8") as fh:
543             json.dump(out, fh, indent=2, sort_keys=True)
544             fh.write("\n")
545         os.replace(tmp, path)
546
547
548     def build_parser() -> argparse.ArgumentParser:
549         p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
550 (offline, CPU).")
551         p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
552         p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
553             help="dir holding <name>_golden_features.json")
554         p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
555 path")
556         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
557 are written")
558         p.add_argument("--iou", type=float, default=DEFAULT_IOU)
559         p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
560             help="F1-like aggregate drop that counts as a regression")
561         p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
562             help="merge_rate rise that counts as a regression")
563         p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
564             help="per-video split/merge-count increase allowed before it's a
565 regression")
566         p.add_argument("--update-baseline", action="store_true",
567             help="snapshot the current numbers as the new baseline instead of
568 checking")
569         p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
570 for tests)")
571         p.add_argument("--no-report", action="store_true", help="skip writing the dated
572 report file")
573         return p
574
575
576     def main(argv: Optional[List[str]] = None) -> int:
577         import logging
578         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
579         log = logging.getLogger("nightly_regression")
580         args = build_parser().parse_args(argv)
581         tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
582 over_seg_tol=args.over_seg_tol)
583         date = args.date or _dt.date.today().isoformat()

```

```

576
577         if not os.path.exists(args.manifest):
578             log.error("manifest not found: %s ? the golden corpus must be present locally.",
args.manifest)
579             return 2
580
581         try:
582             snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
args.features_dir, args.iou)
583         except RuntimeError as e:
584             log.error("%s", e)
585             return 2
586         snapshot["git_commit"] = _git_commit()
587         snapshot["generated_at"] = date
588
589         if args.update_baseline:
590             save_baseline(args.baseline, snapshot, tols)
591             log.info("baseline snapshot written to %s", args.baseline)
592             for cfg, s in snapshot["configs"].items():
593                 agg = s["aggregate"]
594                 log.info(" %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
595                     cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
596                     agg.get("merge_rate") or 0.0)
597             return 0
598
599         baseline = load_baseline(args.baseline)
600         if baseline is None:
601             log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
602             return 3
603
604         result = compare_all(baseline, snapshot, tols, skipped)
605         report = format_report(baseline, snapshot, result, tols, date)
606
607         if not args.no_report:
608             os.makedirs(args.report_dir, exist_ok=True)
609             md_path = os.path.join(args.report_dir, f"{date}.md")
610             json_path = os.path.join(args.report_dir, f"{date}.json")
611             with open(md_path, "w", encoding="utf-8") as fh:
612                 fh.write(report + "\n")
613             with open(json_path, "w", encoding="utf-8") as fh:
614                 json.dump({"date": date, "status": result.overall_status(),
615                     "exit_code": result.exit_code(), "current": snapshot,
616                     "regressions": [f.message() for c in result.comparisons for f in
c.regressions]}},
617                     fh, indent=2, sort_keys=True)
618             log.info("report written to %s", md_path)
619
620         print(report)
621         code = result.exit_code()
622         if code == 1:
623             log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                 sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625         elif code == 4:
626             log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627         else:
628             log.info("nightly regression check PASSED")
629         return code
630
631
632 if __name__ == "__main__":
633     raise SystemExit(main())
634

```

29. assistant (2026-06-22T09:09:01.664Z)

Perfect! Now let me check the baseline and config:

30. user (2026-06-22T09:09:03.597Z)

```
1      {
2      "_doc": "Frozen nightly rally-quality regression baseline. Regenerate with `python
scripts/nightly_regression.py --update-baseline` after an intended improvement or a corpus
change. Tied to the LOCAL golden corpus (not committed).",
3      "configs": {
4      "baseline_off": {
5      "aggregate": {
6      "dend_abs_median": 41.77921047714383,
7      "dstart_abs_median": 42.467243955066174,
8      "f1@0.25": 0.39981006941991765,
9      "f1@0.5": 0.2366145803119009,
10     "merge_rate": 0.6380248744753217,
11     "precision": 0.2577557018251538,
12     "recall": 0.22719343607869802,
13     "segment_count_ratio": 0.5824947574930804,
14     "split_rate": 0.0,
15     "tol_f1": 0.1382064040388172,
16     "tol_precision": 0.14536686157033415,
17     "tol_recall": 0.13445461315285676
18     },
19     "iou": 0.5,
20     "min_crossings": 0,
21     "n": 15,
22     "note": "Pre-R1-flip all-OFF windowing \u2014 the historical floor; pins the value
the milestone buys.",
23     "per_video": {
24     "Badminton_BXH_2": {
25     "f1@0.25": 0.0,
26     "f1@0.5": 0.0,
27     "merge_count": 3,
28     "merge_rate": 1.0,
29     "num_gt": 44,
30     "num_pred": 4,
31     "split_count": 0,
32     "tol_f1": 0.0
33     },
34     "Boxhill_Doubles": {
35     "f1@0.25": 0.10526315789473685,
36     "f1@0.5": 0.03508771929824561,
37     "merge_count": 7,
38     "merge_rate": 0.9534883720930233,
39     "num_gt": 43,
40     "num_pred": 14,
41     "split_count": 0,
42     "tol_f1": 0.0
43     },
44     "GX010128": {
45     "f1@0.25": 0.14925373134328357,
46     "f1@0.5": 0.05970149253731343,
47     "merge_count": 12,
48     "merge_rate": 0.9615384615384616,
49     "num_gt": 52,
50     "num_pred": 15,
51     "split_count": 0,
52     "tol_f1": 0.0
53     },
54     "GX010137": {
55     "f1@0.25": 0.9253731343283583,
56     "f1@0.5": 0.5970149253731343,
```

```

57         "merge_count": 3,
58         "merge_rate": 0.17647058823529413,
59         "num_gt": 34,
60         "num_pred": 33,
61         "split_count": 0,
62         "tol_f1": 0.4477611940298507
63     },
64     "GX010141": {
65         "f1@0.25": 0.5416666666666666,
66         "f1@0.5": 0.3333333333333333,
67         "merge_count": 5,
68         "merge_rate": 0.6551724137931034,
69         "num_gt": 29,
70         "num_pred": 19,
71         "split_count": 0,
72         "tol_f1": 0.3333333333333333
73     },
74     "GX020094": {
75         "f1@0.25": 0.7333333333333334,
76         "f1@0.5": 0.4444444444444445,
77         "merge_count": 3,
78         "merge_rate": 0.15,
79         "num_gt": 40,
80         "num_pred": 50,
81         "split_count": 0,
82         "tol_f1": 0.2888888888888889
83     },
84     "GX030094": {
85         "f1@0.25": 0.5352112676056339,
86         "f1@0.5": 0.28169014084507044,
87         "merge_count": 8,
88         "merge_rate": 0.5609756097560976,
89         "num_gt": 41,
90         "num_pred": 30,
91         "split_count": 0,
92         "tol_f1": 0.14084507042253522
93     },
94     "adarsh_avi_singles": {
95         "f1@0.25": 0.8601036269430052,
96         "f1@0.5": 0.6321243523316062,
97         "merge_count": 5,
98         "merge_rate": 0.10416666666666667,
99         "num_gt": 96,
100        "num_pred": 97,

```

31. assistant (2026-06-22T09:09:10.958Z)

Now let me check the config.json to understand the pipeline structure:

32. user (2026-06-22T09:09:11.846Z)

```

1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
14                     40,

```

```

15         40
16     ],
17     "court_green_upper": [
18         85,
19         255,
20         255
21     ],
22     "court_pixels_min_ratio": 0.05
23 }
24 },
25 "indexing": {
26     "default_segmenter": "gemini",
27     "use_gpu": true,
28     "motion_threshold": 0.08,
29     "min_segment_duration": 3.0,
30     "dead_time_merge_gap": 2.0,
31     "chunk_duration": 90,
32     "chunk_overlap": 10,
33     "detector_model": "fasterrcnn_resnet50_fpn",
34     "detector_score_threshold": 0.5,
35     "yolo_velocity_threshold": 0.15,
36     "yolo_frame_skip": 3,
37     "yolo_tracker": "bytetrack.yaml",
38     "yolo_prefetch_workers": 2,
39     "min_action_window_duration": 2.0,
40     "log_yolo_candidates": true,
41     "store_yolo_candidates": true,
42     "ai_handoff_padding": 2.0,
43     "ai_max_workers": 1,
44     "tracknet": {
45         "wsl_distro": "Ubuntu",
46         "repo_dir": "~/models/TrackNetV4",
47         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48         "conda_env": "TrackNetV4",
49         "weights_path": "",
50         "queue_length": 5,
51         "stage_in_wsl": true,
52         "wsl_stage_dir": "~/clips",
53         "timeout_sec": 1800,
54         "velocity_threshold": 0.02
55     },
56     "wasb": {
57         "wsl_distro": "Ubuntu",
58         "repo_dir": "~/models/WASB-SBDT",
59         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60         "conda_env": "wasb",
61         "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
62         "sport": "badminton",
63         "wsl_stage_dir": "~/clips",
64         "timeout_sec": 1800,
65         "velocity_threshold": 0.02
66     }
67 },
68 "output": {
69     "default_highlights_name": "highlights.mp4",
70     "db_name": "sports_indexer.db"
71 },
72 "stitching": {
73     "begin_padding": 0.5,
74     "end_padding": 0.5,
75     "use_cache": false,
76     "min_shot_count": 1,
77     "watermark": {
78         "enabled": true,

```

```

79         "text": "Generated by Khelsutra.guru",
80         "position": "bottom-right",
81         "opacity": 0.85,
82         "logo_path": "",
83         "font_path": ""
84     }
85 }
86 }

```

33. assistant (2026-06-22T09:09:15.308Z)

Now let me check for any cached golden features:

34. user (2026-06-22T09:09:16.452Z)

```

-rw-r--r-- 1 avidu 197609 3.1K Jun 17 15:06 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/Badminton_BXH_2_golden_features.json
-rw-r--r-- 1 avidu 197609 18K Jun 18 12:26 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/Boxhill_Doubles_golden_features.json
-rw-r--r-- 1 avidu 197609 6.3K Jun 16 22:32 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010128_golden_features.json
-rw-r--r-- 1 avidu 197609 16K Jun 18 10:22 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010137_golden_features.json
-rw-r--r-- 1 avidu 197609 8.1K Jun 18 07:04 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/GX010141_golden_features.json
-rw-r--r-- 1 avidu 197609 28K Jun 18 12:26 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/GX020094_golden_features.json
-rw-r--r-- 1 avidu 197609 9.6K Jun 17 15:06 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/GX030094_golden_features.json
-rw-r--r-- 1 avidu 197609 129K Jun 14 11:54 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/adarsh_avi_singles_golden_features.json
-rw-r--r-- 1 avidu 197609 62K Jun 14 11:54 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/gbaaddy_golden_features.json
-rw-r--r-- 1 avidu 197609 16K Jun 14 11:54 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/kushagra_singles_golden_features.json
-rw-r--r-- 1 avidu 197609 64K Jun 14 11:54 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/largetest_doubles_golden_features.json
-rw-r--r-- 1 avidu 197609 827 Jun 17 13:56 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_1_golden_features.json
-rw-r--r-- 1 avidu 197609 3.2K Jun 16 19:45 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_2_golden_features.json
-rw-r--r-- 1 avidu 197609 43K Jun 14 11:54 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/mahadevpura_singles_golden_features.json
-rw-r--r-- 1 avidu 197609 11K Jun 14 11:54 /c/Users/avidu/Projects/badminton-highlight-
indexer/output/testlarge_short_golden_features.json

```

35. assistant (2026-06-22T09:09:18.424Z)

Perfect! Now let me read one golden features file to understand its structure:

36. user (2026-06-22T09:09:19.041Z)

```

1  {
2      "name": "mahadevpura_1",
3      "fps": 59.94,
4      "frame_width": 1920.0,
5      "candidates": [
6          {
7              "start": 0.016683350016683352,
8              "end": 174.29095762429097,
9              "active_frame_count": 4230,
10             "peak_velocity": 9.35547211247393,
11             "mean_velocity": 0.20538627104988216,
12             "track_id_count": 1,

```

```

13         "chunk_index": null
14     }
15 ],
16 "gts": [
17     [
18         18.156,
19         24.156
20     ],
21     [
22         34.155,
23         38.155
24     ],
25     [
26         43.905,
27         48.155
28     ],
29     [
30         54.905,
31         61.156
32     ],
33     [
34         70.655,
35         78.655
36     ],
37     [
38         83.406,
39         96.656
40     ],
41     [
42         103.406,
43         114.656
44     ],
45     [
46         122.156,
47         125.906
48     ],
49     [
50         133.406,
51         136.406
52     ],
53     [
54         142.406,
55         146.154
56     ]
57 ]
58 }

```

37. assistant (2026-06-22T09:09:21.285Z)

Excellent! Now let me compile the comprehensive analysis. Let me also check the PowerShell wrapper script:

38. user (2026-06-22T09:09:22.401Z)

```

total 100K
-rw-r--r-- 1 avidu 197609 1.4K Jun 22 14:36 README.md
-rwxr-xr-x 1 avidu 197609 4.5K Jun 22 14:36 bundle_ui.sh
-rw-r--r-- 1 avidu 197609 5.7K Jun 22 14:36 doctor.py
-rwxr-xr-x 1 avidu 197609 10K Jun 22 14:36 install.py
-rw-r--r-- 1 avidu 197609 3.6K Jun 22 14:36 nightly_regression.ps1
-rw-r--r-- 1 avidu 197609 31K Jun 22 14:36 nightly_regression.py
-rw-r--r-- 1 avidu 197609 3.4K Jun 22 14:36 register_nightly_task.ps1
-rw-r--r-- 1 avidu 197609 8.4K Jun 22 14:36 run_phase3_eval.py
-rw-r--r-- 1 avidu 197609 7.5K Jun 22 14:36 run_process_and_stitch.py
-rwxr-xr-x 1 avidu 197609 7.5K Jun 22 14:36 uninstall.py

```

39. user (2026-06-22T09:09:25.080Z)

```
1      <#
2      .SYNOPSIS
3          Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5      .DESCRIPTION
6          Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
7          (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline.
Writes a
8          dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker,
then exits
9          with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11         GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present
yet (e.g.
12         before this PR is merged into the checkout the task points at) ? so a registered
task never
13         errors in the interim and self-activates once the files land.
14
15         Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE
the repo on
16         this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18     .PARAMETER RepoRoot
19         Repo root to test. Default: the parent of this script's directory.
20
21     .PARAMETER PythonExe
22         Python interpreter. Default: 'python'.
23
24     .PARAMETER Pull
25         If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26         that may hold a sibling session's WIP).
27     #>
28     [CmdletBinding()]
29     param(
30         [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31         [string]$PythonExe = 'python',
32         [switch]$Pull
33     )
34
35     $ErrorActionPreference = 'Continue'
36     $date = Get-Date -Format 'yyyy-MM-dd'
37     $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38     $logDir = Join-Path $reportDir 'logs'
39     New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40     $log = Join-Path $logDir "$date.log"
41     $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
42
43     function Write-Status([string]$status, [int]$code, [string]$detail) {
44         $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45         "$stamp status=$status exit=$code $detail" | Out-File -FilePath $statusFile
-Encoding utf8
46     }
47
48     $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49     $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
50
51     # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
52     if (-not (Test-Path $checker)) {
53         "$date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
| Tee-Object -FilePath $log
```

```

54     Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
55     exit 0
56 }
57 if (-not (Test-Path $manifest)) {
58     "[date] golden manifest not present at $manifest ? skipping (corpus absent on this
machine)." | Tee-Object -FilePath $log
59     Write-Status 'SKIPPED' 0 'golden corpus absent'
60     exit 0
61 }
62
63 Set-Location $RepoRoot
64 if ($Pull) {
65     "[date] git pull --ff-only" | Tee-Object -FilePath $log -Append
66     & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
67 }
68
69 $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
70 "[date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
-Append
71
72 & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
73 $code = $LASTEXITCODE
74
75 switch ($code) {
76     0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
77     1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
78         "[date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
Object -FilePath $log -Append }
79     4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
80     default { Write-Status 'ERROR' $code "checker error ? see $log" }
81 }
82
83 exit $code
84

```

40. assistant (2026-06-22T09:09:27.113Z)

Perfect! Now let me create a comprehensive summary. Let me also check the README for any additional details:

41. user (2026-06-22T09:09:28.463Z)

```

1     # scripts/
2
3     **Standalone run-drivers / one-off entry CLIs** ? scripts with a `__main__` that you
*run*, not
4     *import*. Run as `python scripts/<name>.py`.
5
6     What belongs here (vs. elsewhere ? see `docs/CODE_MAP.md` §0 for the full table):
7     - ? a CLI that orchestrates a pipeline/eval run end-to-end (e.g. `run_phase3_eval.py`,
`run_process_and_stitch.py`)
8     - ? **importable eval library** (metrics, calibration, a scorer) ? `backend/eval/`
9     - ? an **ops/maintenance tool** run as `python -m tools.x` ? `tools/`
10    - ? a **throwaway repro / debug** script ? `scratch/` (gitignored)
11    - ? anything imported by app code ? it's library, put it under `backend/`
12
13    If a script needs `load_dotenv()` / `sys.path` setup before importing `backend`, add
14    `scripts/<name>.py" = ["E402"]` to `ruff.toml` (import-order is load-bearing there).
15
16    Grandfathered at the repo root (referenced by CI/packaging/tests ? left in place):
17    `run_all_tests.py`, `run_regression.py`, `run_eval.py`. **New** run-drivers go here, not
the root.
18
19    `doctor.py` here is the diagnostics/bootstrap script (FFmpeg + CUDA checks, default-
config

```

```
20     init, `pip install -r requirements.txt`). It was formerly the root `setup.py`; it was
renamed
21     and moved here so it no longer shadows the PEP 517 build system now that
`pyproject.toml` exists.
22     Run it as `python scripts/doctor.py` or `indexer --setup`.
23
```

42. assistant (2026-06-22T09:09:31.669Z)

Now let me create the final structured output with all the gathered data:

43. user (2026-06-22T09:10:09.262Z)

Structured output provided successfully

44. assistant (2026-06-22T09:10:17.038Z)

Done. Structured data returned via StructuredOutput tool as required by the workflow harness.
The nightly regression system is fully mapped:

```
**CORE INFRASTRUCTURE READY:**
- Tripwire: `scripts/nightly_regression.py` re-scores cached WASB trajectories against 3 configs
(DEFAULT/MILESTONE/BASELINE_OFF)
- Baseline frozen at: `eval_baselines/nightly_baseline.json` (n=15 videos)
- Manifest: `output/human_lovo_manifest.json` with trajectory + label + cached feature paths
- Windows task wrapper: `scripts/nightly_regression.ps1` (graceful no-op if corpus absent)
```

```
**GOLDEN VIDEO TRIO (SHORT, CACHE-COMPLETE):**
```

1. ****GX010141**** ? 6.42 min, 29 rallies, trajectory + golden-features cached (2026-06-17)
2. ****GX010137**** ? 7.52 min, 34 rallies, trajectory + golden-features cached (2026-06-17)
3. ****mahadevpura_1**** ? 2.90 min, 10 rallies, trajectory + golden-features cached (2026-06-16)

All have locally-cached trajectories (no GPU needed) + populated gts/candidates in golden-
feature JSON. Expected nightly runtime: 15?30 sec pure CPU, fully deterministic.